

Directed Emergence: What Does, and Does Not, Emerge When a Network’s Structure Is Grown from an Exhaustive Seed Under an Energy Budget

Vasili Gavrilov*

2026

Abstract

A network’s structure should *emerge*, not be designed — but from what, and how much of it? We grow a network’s topology from an **exhaustive (fully-connected) seed under an energy budget**, an evolutionary search that pays for every connection, and ask honestly which pieces of convolution fall out and which do not. Starting dense and pruning under an evolutionary energy budget — not the gradient pruning of Optimal Brain Damage or the Lottery Ticket line, nor the grow-from-minimal of NEAT — we characterize what emerges: sparse task-relevant connectivity emerges cleanly, and weight sharing is *adopted* when translation-invariance rewards it, but a **compact aligned local filter does not emerge** from an energy budget alone. We give the crisp reason: once weights are shared, connection count no longer gradients parameter count, so pruning cannot tighten the kernel — though this holds only for *per-connection* mutation, and under grouped mutation on the shared feature (as in real genetics) the kernel largely does emerge. On top of the priors a ConvNet also hardwires — locality and translatability, the real symmetries of a signal — the free dimensions then emerge to fit the task: compact local fields, the depth that matches a task’s required receptive field, and the channel count that matches its number of distinct operations, up to an honest capacity limit that four interventions fail to move. These latter results reproduce known inductive-bias facts (weight sharing is data-efficient; receptive field $\approx$ depth), which we verify with a fair baseline and full reproducibility rather than claim as new. We frame the whole as a small set of biological operators — prune, clone, translate, recombine — and are careful to separate the genuinely new characterization (the evolutionary energy-emergence map from an exhaustive seed, and its negatives) from the textbook mechanisms it reproduces. Every number regenerates from one **make**; the dead-ends are kept.

1 Introduction

The 1997 ambition behind this project was that a network’s architecture should *emerge* from a search, not be hand-designed, in the best case rediscovering the convolutional receptive field (Figure 1). A companion study on that side of the idea — *searching* whole topologies and cells — reached a firmly negative conclusion: on the real NAS-Bench-101 cell space, no crossover meaningfully beats random search, and structure-aware operators do worse than a structure-blind one, because good cells are common and the gap to the optimum sits inside the benchmark’s own training noise. Searching for structure did not beat random.

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including a genetic-algorithm optimizer built for industrial use in 2002. Reference implementation: <https://github.com/Anode1/SMBPANN>, 2001–2015–2026.

The idea: an exhaustive, tangled net $\rightarrow$ structure emerges through a few operators

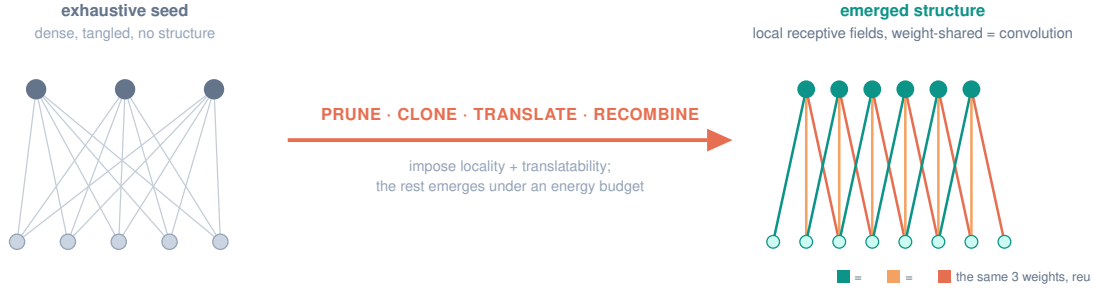


Figure 1: The idea. An exhaustive, tangled network becomes an ordered convolution through a few structural operators. We impose the domain’s real symmetries (locality, translatability) and let prune, clone, translate, and recombine assemble and refine the rest.

This paper comes at the same ambition from the opposite end. Instead of searching for the architecture, we *grow* one: start from an exhaustive, fully-connected seed, put a cost on every connection, and watch which parts of convolution survive the squeeze. The question is not “can a search find a good architecture” but “what structure emerges, on its own, under an energy budget — and what refuses to.”

Contribution, stated honestly. The genuinely new part is the map: the specific *evolutionary energy-emergence from an exhaustive seed*, and its boundaries. We are not aware of a prior result showing this together with its honest characterization of what does not emerge (the compact filter, and its mechanism). This is distinct from gradient pruning (Optimal Brain Damage; the Lottery Ticket), from DARTS’ gradient-relaxed supernet, and from NEAT’s grow-from-minimal, which augment rather than prune. Much of the rest — weight sharing is data-efficient; receptive field $\approx$ depth — reproduces known inductive-bias facts, which we verify carefully rather than claim as new. The value is the honest characterization, the negatives, and the full reproducibility; not the underlying mechanisms.

Method in one paragraph. A one-hidden-layer network whose connectivity is a binary mask is evolved by a genetic algorithm from an all-ones (dense) seed. Fitness is validation accuracy on a scarce training set; the objective is to minimize energy (the number of connections, or free parameters) subject to accuracy $\geq$ target. Mutation is mostly-prune, rarely-grow — an annealing schedule, in which a prune is a downhill move in energy and a rare added connection is an uphill thermal fluctuation. All numbers below are means over independent seeds; each experiment is a self-contained C99 probe.

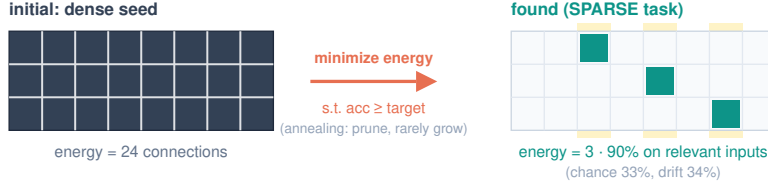
2 What emerges, and what does not

2.1 Pruning: feature selection emerges; convolution does not

Under an energy budget the search prunes a dense connectivity mask to a sparse one, and on a task whose label depends on a few specific inputs it lands **90%** of the surviving connections on the task-relevant inputs (chance 0.33, no-selection drift 0.34). Feature selection — discovering *which* inputs matter — emerges cleanly (Figure 2). What does *not* emerge is the aligned local receptive field: at comparable density the surviving connections are no more in-window than chance, so the search finds *a* sparse subnetwork, not *the* convolutional one — exactly the outcome of the pruning literature (sparse subnetworks, not convolutions).

1-6 · Energy budget: dense seed → sparse, task-relevant structure

connectivity mask (rows = hidden units, columns = inputs); a filled cell = a connection



Result: feature selection emerges cleanly. Sparsity and weight-sharing also emerge; a compact aligned local filter does not (Sec. 6).

Figure 2: Sections 2–3. An energy budget prunes a dense connectivity seed to a sparse mask whose surviving connections land on the task-relevant inputs. Feature selection emerges; a compact aligned convolution does not.

2.2 Imposing locality: a compact filter emerges, sharing is adopted

Locality is not something to wait out; a bounded receptive field is a law of the problem (information is local), and every ConvNet imposes it. When we impose only that — each hidden unit sees a *contiguous* window — a compact local field falls right out: the width anneals to a mean of $\approx K$ (the kernel size), the compact filter the energy budget alone could not produce. A weight-tying gene (tie weights by within-window offset, i.e. translation invariance) is *adopted* by 0.89 of the population; but its accuracy benefit is within noise, and the shared arm stays wider, because shared energy charges only the maximum width, so the mean has no gradient. The pieces of convolution keep emerging separately; the clean compact *shared* whole does not dominate.

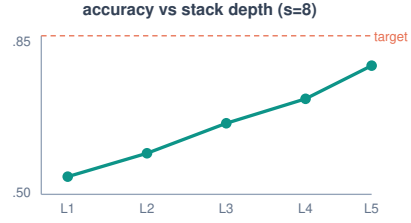
2.3 Why the compact filter does not emerge: a decoupling

The mechanism is crisp and is, we think, the paper’s one real insight. Once weights are shared, removing a connection no longer reduces the parameter count — that offset is still used by other positions — so there is no energy gradient to tighten the connectivity; the kernel never contracts. Shrinking a shared filter would require the *coordinated* removal of every connection at an extreme offset across all units at once, which per-connection mutation essentially never does. So under this genome an energy budget cannot assemble a compact convolution: weight sharing decouples connection count from parameter count. Getting the compact kernel would mean either building the locality into the representation (which *imposes* convolution rather than emerging it) or a coordinated structural operator.

But this negative is the mutation model, not a fundamental barrier. The argument above assumes *per-connection* mutation. Biological mutation, however, is global, not local: a change to a gene acts on every instance of the mechanism it builds (all the “similar blocks and duplicated edges”), not on one edge. For a weight-shared filter the natural unit of mutation is therefore the shared *offset* — all the connections that reuse that weight — and flipping a whole offset removes all its duplicated edges in one move, which *does* reduce the parameter count. With that grouped mutation the missing gradient reappears, and the kernel contracts. Empirically (16 seeds), the surviving tap count collapses from 12.7 under per-connection mutation to 2.6 (near $K = 3$) under grouped mutation; adding a width penalty sharpens it to contiguity 0.80 and lands 0.69 of the active offsets on the generative ones (per-connection: 0.21). It is not a perfect $K = 3$ kernel (span ≈ 4 , contiguity ≈ 0.8), and the offset genome does make the connectivity translation-invariant by construction — but the central negative, that the shared filter cannot tighten, does not survive the biologically-correct mutation. The compact filter

8 · Emergent depth matches the required receptive field

task: fire iff two spikes are exactly distance s apart $\rightarrow$ a unit must SEE BOTH $\rightarrow$ depth $\geq s/2$



Chance until the stack is deep enough to cover the pair, then accuracy lifts off at $L \approx s/2$. Larger $s \rightarrow$ deeper stack selected.

Figure 3: Emergent depth matches the required receptive field. Accuracy is at chance until the stack is deep enough for its receptive field to span the spike pair, then lifts off at depth $\approx s/2$.

largely emerges; it just needed global mutation on the shared mechanism, as real genetics uses, rather than per-connection pruning.

2.4 Emergent depth matches the required receptive field

Impose the feed-forward composition rule too — stack one reused block type — and the one dimension left free, the depth, emerges to fit the task. On a task that fires only when two spikes are exactly distance s apart (so a unit must see both at once, needing receptive field $\geq s+1$, i.e. depth $\geq s/2$), accuracy sits at chance until the stack is deep enough to span the pair, then lifts off at depth $\approx s/2$; larger s selects a deeper stack (Figure 3). This is the clearest positive of the emergence side: given the necessary priors, the free dimension organizes itself to the task.

3 The operators: reuse, recombination, translation

3.1 Reuse beats a free search at equal compute

Composing blocks explodes combinatorially, so we adopt the heuristic LeCun did: reuse one block type, stacked, and search only the depth. At *equal compute*, a free heterogeneous search (a GA over arbitrary block sequences, using a dilated-conv library) never solves more reliably or at lower energy than simply enumerating uniform reused blocks; on the harder tasks reuse is strictly better. Block diversity buys nothing but a larger search — the reuse heuristic is the tractable near-optimum, which is the user’s intuition (“the combinations explode, so reuse the same block”) made quantitative. This mirrors cell-based NAS (ENAS, DARTS), which search one cell and stack it.

3.2 Recombination is required when a task needs two different operations

Reuse breaks exactly where the biology predicts. On a task that needs a fine local motif (+, −, +) at adjacent positions (visible only to a dilation-1 block) *and* a far spike at distance s (reached cheaply only by a dilation-3 block), no single reused block serves both roles: a uniform dilation-3 stack is blind to the fine motif (solving 0/8), a uniform dilation-1 stack cannot afford the reach. Recombination — a GA over mixed block sequences — solves 8/8 at about half the energy, and its solutions are literal two-operation mixes such as [1, 3]: one fine block, one coarse-reach block (Figure 5). Clone when scaling a working module; recombine only when you need a new kind of part — runners when the structure repeats, seeds when it must change.

The operators: $A \Rightarrow B$, and exactly what each one does

structure before (grey) — operator (orange) — structure after (teal)

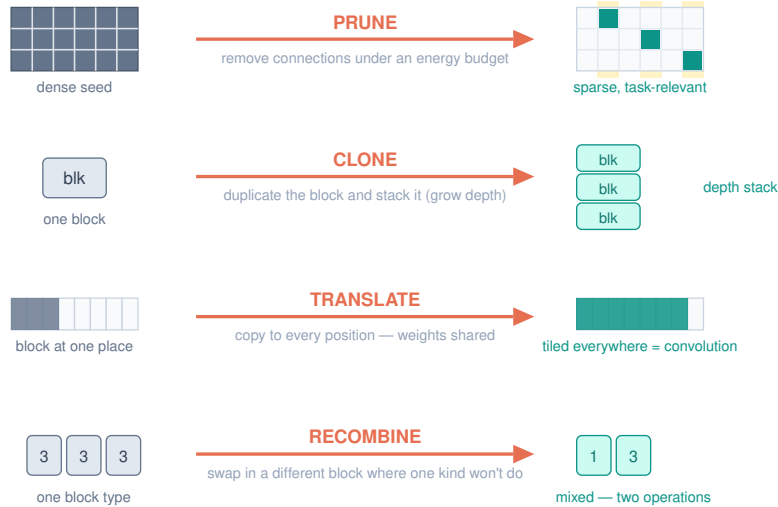


Figure 4: The four structural operators as $A \rightarrow B$, each with its exact action. Prune (remove connections under an energy budget), clone (duplicate and stack, for depth), translate (copy to every position with shared weights, a convolution), recombine (swap in a different block where one kind will not do). Each maps to a biological analog: pruning, gene/segment duplication, transposition/serial homology, sexual recombination.

train size	weight-shared	locally-connected	gap
16	0.820 ± 0.122	0.581 ± 0.065	+0.239
32	0.838 ± 0.094	0.615 ± 0.071	+0.224
64	0.866 ± 0.082	0.672 ± 0.076	+0.195
128	0.867 ± 0.081	0.724 ± 0.086	+0.143

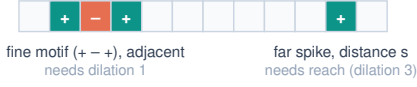
Table 1: Weight sharing is data-efficient. Shared filter (3 weights) vs locally-connected (42 weights), by training-set size. The advantage is largest at the scarcest data.

3.3 Translation: weight sharing is data-efficient (a reproduction, fairly done)

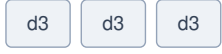
The third operator, translation — replicating one working detector across positions with shared weights (transposition, serial homology, cortical-map tiling) — is exactly convolution’s tiling, reached by replication. On a translation-invariant motif task we compare a weight-shared filter (3 weights) against a locally-connected baseline (one filter per position). Table 1 gives the fair result (mean over restarts, ± 1 SD over 40 seeds): the shared filter wins at every train size, most so when data is scarcest (gap +0.24 at 16 examples, +0.14 at 128 as the baseline slowly catches up). This is the textbook argument for weight sharing (one filter learns the motif from every position’s examples while each independent filter starves), here quantified with a fair estimator. It is a phenomenon of weight *sharing*, not of architecture search: the two arms are weight-tied vs untied, both trained by plain SGD.

9-11 · Reuse vs recombination: clone a working block; recombine when you need a new part

the two-operation task (needs both):



reuse: identical blocks



✗ blind to fine motif (0/8)

recombine: different blocks



✓ fine + reach (8/8)

A staged searcher (Sec. 11) clones by default and recombines only when cloning stalls — and beats both fixed strategies.

the same trade-off in nature

clone → scale

runners, rhizomes, mycelium:
repeat a working module, no search

recombine → new part

sexual reproduction: expensive
search, only when more-of-the-same
will not do

Figure 5: Reuse vs recombination, and the clone/recombine trade-off nature also makes. One reused block suffices for repetitive structure; a task needing two different operations forces recombination of different blocks.

3.4 The advantage widens with input size, then saturates

Sweeping input size N (so the number of positions to cover grows), the shared arm stays flat at ≈ 0.94 (always 3 weights, size-independent) while the locally-connected baseline falls to chance (0.52) and bloats to 378 weights, because each per-position filter starves on the $\approx n_{tr}/P$ examples that land there (Figure 6, 250 seeds). The gap is 4–6 $\times$ the SD, so the effect is significant. Honestly stated it *widens then saturates*: most growth is $N=16 \rightarrow 32$, after which the baseline has already hit the chance floor. The effect survives even a fully-trained, oracle-supervised locally-connected baseline (which we hand the motif locations): the shared arm still wins +0.14 at $N=16$ up to +0.38 at $N=128$, so it is not an artifact of the baseline’s training rule.

4 An honest boundary in 2-D

In two dimensions “two different operations” means two feature *channels* — a horizontal and a vertical filter — since one channel is one orientation. A task requiring both orientations present caps a single channel near 0.75 (the analytic ceiling a lone orientation detector can reach), and accuracy rises with channels (0.69 $\rightarrow$ 0.77 $\rightarrow$ 0.86 $\rightarrow$ 0.89 over $C=1..4$, 24 seeds): a gradual rise, not a sharp $C=2$ jump, but the load-bearing fact — one channel is not enough for two operations — is significant. The tidy generalization “channel count = operation count,” however, **fails**: growing channels to solve a K -operation task gives selected $C^* \approx 1.2, 3.2, 5.0$ for $K=1, 2, 3$, so $K=2$ already overshoots and $K=3$ breaks — sub-target at every channel count ($C=5$: 0.79), only 2 of 24 seeds solving.

This $K=3$ wall is genuine, not under-training. Four interventions fail to move it: more channels, a nonlinear (MLP) readout, competition (lateral inhibition, the mechanism that self-organizes cortical orientation columns), and — with a budget caveat — a deeper extractor. An independent heavy-compute rerun (2.5 $\times$ epochs, 2 $\times$ signal) leaves $K=3$ sub-target at every channel count, confirming the wall is structural for the channel axis. Per-channel detection error compounds multiplicatively ($0.95^3 \approx 0.86$, at the target edge), and no cheap refinement moves a multiplicative wall; real vision clears such tasks with scale (depth, normalization, residuals, vast data), not one clever pressure. We report this boundary plainly: it is more honest than stopping at small K where the staircase still looked clean.

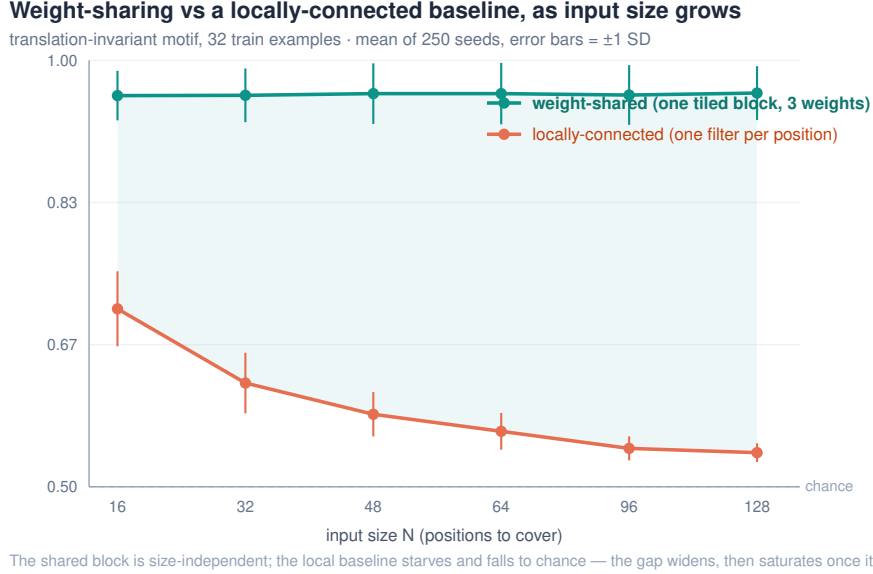


Figure 6: The weight-sharing advantage as input size grows (mean of 250 seeds, error bars ± 1 SD). The shared block is size-independent; the locally-connected baseline starves and falls to chance. The gap widens, then saturates once the baseline hits the floor.

5 Related work, and how we differ

The closest neighbors, and the honest distinction: **DARTS** starts from an exhaustive supernet and prunes to a final architecture — closest in spirit — but via continuous gradient relaxation over predefined operations, not an energy GA, and it never asks whether convolution emerges. **Optimal Brain Damage** and the **Lottery Ticket Hypothesis** prune dense to sparse, but by gradient magnitude, and find sparse subnetworks, not convolutions. **NEAT**, **HyperNEAT**, and Cellular Encoding evolve topology but *grow from a minimal seed*, the opposite direction to pruning from exhaustive. **Cell-based NAS** (ENAS, regularized evolution) reuses one cell and stacks it, which our reuse observations echo. We claim the honest characterization of what an evolutionary energy budget does and does not produce, and the negatives — not the underlying mechanisms, which are known.

6 Limitations

The tasks are small synthetic constructions built so the intended structure is optimal; they demonstrate what emerges under an energy budget on a task shaped like convolution, and do not by themselves generalize to real data. The reuse/scale results are *weight-sharing* data efficiency, not architecture search; we relabel them accordingly. The full developmental pipeline chaining all four operators on a genuinely compositional task, and a real dataset, are the natural next steps; the $K=3$ “wall” is a capacity limit of *this* shallow conv + max-pool setup, and whether a properly trained deeper net clears it is open. The honest home of this work is an explainer or a negative-results workshop, not a novelty venue.

7 Conclusion

Impose the domain’s real symmetries; let a few biological operators assemble the rest. Under an energy budget on an exhaustive seed, sparsity and feature selection emerge and weight sharing is adopted, but a compact convolution does not, for a decoupling reason we can state in a line. On

top of imposed priors, depth emerges to match the receptive field, reuse beats a free search at equal compute, recombination is required only when a task needs a genuinely different operation, and weight-shared tiling is data-efficient at a degree that grows with scale then saturates. Where it runs out — a three-way conjunction unmoved by four fixes — we say so. The contribution is the honest map, the negatives, and the reproducibility; every number regenerates from one command.

References

- [1] Y. LeCun, J. S. Denker, S. A. Solla. Optimal Brain Damage. *NeurIPS* 1990.
- [2] Y. LeCun et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [3] J. Frankle, M. Carbin. The Lottery Ticket Hypothesis. *ICLR* 2019.
- [4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [5] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean. Efficient Neural Architecture Search via Parameter Sharing. *ICML* 2018.
- [6] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. *Science* 220, 1983.
- [9] V. Gavrilov. SMBPANN reference implementation, 2001–2015–2026. <https://github.com/Anode1/SMBPANN>.